Real-Time Wildfire Detection from Satellite Feeds — End-to-End Pipeline

Introduction

Building an automated, real-time wildfire detection system from satellite feeds is a complete end-to-end engineering pipeline: data acquisition, orbital physics, computer vision, and operational deployment.

Phase 1: The Problem Formulation (Semantic Segmentation)

Before writing any code or downloading data, you must define what the neural network is mathematically doing.

In computer vision, tasks fall into three main categories:

- **Classification:** "Is there a fire somewhere in this image? (Yes / No)."
- **Object Detection (YOLO / Faster R-CNN):** "Draw a bounding box around the fire."
- **Semantic Segmentation:** "Classify every individual pixel in the image grid as either Class 0 (No Fire) or Class 1 (Active Fire)."

Wildfire monitoring requires **Semantic Segmentation**.

A single geostationary pixel covers roughly 3 km × 3 km on the ground. A bounding box is meaningless at this scale. You need a model that takes a 2D multi-spectral grid of Northern Algeria and outputs a probability mask of the exact same height and width, where every pixel value is a continuous number between 0.0 and 1.0 indicating fire probability.

```
Input Tensor: [C, H, W]              Neural Network            Output Mask: [1, H, W]
C = Thermal & Spectral Bands            (U-Net)                Values: 0.0 to 1.0
H, W = Spatial Grid of N. Algeria                          (Probability per pixel)
```

Phase 2: Where to Get the Data

You need two distinct datasets: satellite imagery (inputs) and fire labels (targets).

Input Satellite Data
EUMETSAT Meteosat SEVIRI / MTG
- Geostationary (Every 5 min)
- Coarse resolution (~3{4 km)

Ground Truth Labels
NASA FIRMS (VIIRS)
- Polar LEO (375 m resolution)
- Spatial truth reference

Spatial & Temporal
Alignment Pipeline

Training Dataset Pairs
(Tensor X, Binary Mask Y)

1. Satellite Imagery (Your Inputs)

- **Provider:** EUMETSAT Data Store (data.eumetsat.int).
- **Cost:** Free for academic, personal research, and non-commercial development.
- **Product:**
  - Primary (Stable & Historical): Meteosat SEVIRI Rapid Scan Service (RSS). Scans Northern Africa and Europe every 5 minutes.
  - Next-Gen: MTG-I FCI Level 1C (Meteosat Third Generation).
- **Channels you must download:** You do not download normal RGB photography. You need raw calibrated infrared radiometer bands:
  - IR 3.9 μm (Mid-Wave Infrared): Highly sensitive to high-temperature blackbody emissions (600 K − 1200 K).
  - IR 10.8 μm (Thermal Long-Wave Infrared): Measures general ambient ground surface and cloud-top temperatures (250 K − 320 K).
  - VIS 0.6 μm (Visible Red): Daytime reference to evaluate clouds and surface reflectance.

2. Ground-Truth Labels (Your Targets)

Because you cannot walk through forests to log every fire by hand, you use high-resolution polar satellites as your "teacher":

- **Provider:** NASA FIRMS (Fire Information for Resource Management System).
- **Download:** FIRMS Archive Download tool (firms.modaps.eosdis.nasa.gov).
- **Format:** Download CSV files of VIIRS 375m (Suomi-NPP & NOAA-20/21) active fire detections over Algeria covering fire seasons (June to September) from 2020 through 2024.
- **Data Fields:** Each record gives you exact GPS coordinates (latitude, longitude), acq_date, acq_time, confidence, and frp (Fire Radiative Power in Megawatts).

Phase 3: Data Preprocessing & Alignment

This is the most critical technical step: turning raw satellite files and GPS points into arrays your neural network can train on.

Do You Process the Whole Earth or Small Pieces?

You do not process the entire Earth image (the "Full Disk").

A full Meteosat Earth disk is 3712×3712 pixels, covering half the planet. 98% of it is irrelevant ocean, Europe, and southern Africa.

Northern Algeria (from the Mediterranean coast down past the Tell Atlas and Aurès Mountains: Latitude 34.0°N to 37.5°N, Longitude 2.0°W to 8.5°E) represents a compact grid of approximately 250×120 pixels at SEVIRI's native resolution.

This small crop fits directly into GPU memory, allowing fast training and low latency.

The Construction Pipeline

For every historical timestamp where VIIRS detected fires in Algeria:

1. Download the corresponding 5-minute Meteosat file.
2. **Convert Digital Counts to Physical Units:** Raw satellite data comes as 10-bit or 12-bit integer digital counts. Use the calibration coefficients stored inside the file metadata to convert these counts into spectral radiance, then apply the inverse Planck function to compute Brightness Temperature in Kelvin (T_b).
3. **Crop to the Northern Algeria Bounding Box:** Use Python's `pyresample` or `satpy` library to extract your geographic bounding box onto an equidistant grid.
4. **Compute the Differential Feature:** Create a derived channel:

   $$\Delta T = T_{3.9} - T_{10.8}$$

   In normal non-burning forest, ΔT is typically between 0 K and 10 K. Over an active fire, the 3.9 μm band spikes dramatically while 10.8 μm rises only slightly, causing ΔT to jump to 20 K − 60 K.
5. **Construct the Ground Truth Binary Mask:** Create a 2D zero-filled numpy array of shape $(120, 250)$. For every FIRMS VIIRS fire coordinate that fell inside that 5-minute window, map its latitude/longitude to the corresponding pixel index (y, x) in your cropped Meteosat grid and set that pixel to 1.0.

Phase 4: Model Architecture (Why U-Net?)

For this task, a U-Net is the industry standard architecture.

```
Input: [B, 3, 120, 256]


[Conv 3x3 + ReLU] x2   Skip Connection  [Concat + Conv 3x3] x2
```

```
[MaxPool 2x2]                                                    [UpSample 2x2]

[Conv 3x3 + ReLU] x2   Skip Connection  [Concat + Conv 3x3] x2

[MaxPool 2x2]                                                    [UpSample 2x2]

[Conv 3x3 + ReLU] x2   Skip Connection  [Concat + Conv 3x3] x2

[MaxPool 2x2]                                                    [UpSample 2x2]

   [Bottleneck Conv]


                                                          [Conv 1x1 + Sigmoid]


                                                          Output: [B, 1, 120, 256]
```

Why U-Net Works Here

A standard classification network downsamples an image repeatedly using pooling layers until it outputs a single class vector. In downsampling, fine spatial location is destroyed.

Because early-stage wildfires might only affect a single pixel (1×1) in your satellite grid, downsampling the image by a factor of 16 (2^4) would blur that fire pixel out of existence.

U-Net solves this using **Skip Connections**:

- The **Encoder** (Left side) extracts high-level semantic context (e.g., "this entire regional patch is an arid summer landscape with high ambient heat").
- The **Decoder** (Right side) restores the spatial dimensions back to original resolution.
- The **Skip Connections** (Horizontal arrows) copy high-resolution spatial feature maps directly from the encoder stages across to the decoder stages. This gives the network both the broad geographic context and the exact pixel coordinate of the thermal spike.

Phase 5: Metrics & The "Accuracy" Trap

In machine learning for wildfire detection, never use standard classification accuracy.

The Math of Class Imbalance

Consider your cropped grid of Northern Algeria:

$$\text{Total Pixels} = 120 \times 250 = 30,000 \text{ pixels}$$

During an extreme fire day, there might be 30 active fire pixels across the country. The remaining 29,970 pixels are non-fire:

$$\text{Non-fire percentage} = \frac{29,970}{30,000} = 99.9\%$$

If a model predicts 0 (No Fire) for every single pixel on Earth without looking at the input, its accuracy will be 99.9%. It will appear mathematically near-perfect while failing to detect every single wildfire.

The Metrics That Actually Matter

Metric	**Formula**	**What It Measures**	**Target Range**
Recall (Sensitivity)	$\frac{TP}{TP+FN}$	Out of all real fires burning on the ground, what percentage did the model catch?	$85\% - 95\%$ (Missing a real fire is life-threatening)
Precision	$\frac{TP}{TP+FP}$	When the model raises a red flag, how often is it actually a real fire vs. a false alarm?	$70\% - 85\%$
F1-Score / Dice	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	The harmonic mean balancing false alarms against missed detections.	> 0.75
IoU (Jaccard Index)	$\frac{TP}{TP+FP+FN}$	Overlap area between predicted fire mask and real fire mask.	> 0.65

Phase 6: How to Guarantee the Model Is Not Overfitting

Overfitting in remote sensing is fundamentally different from overfitting on regular photo datasets (like ImageNet). If you train carelessly, your model will score high marks during evaluation but fail completely in real-world deployment.

1. Spatial-Temporal Data Splitting (Crucial)

Never use random cross-validation (like `train_test_split(shuffle=True)`).

If a fire burns in Tizi Ouzou on August 10, 2021 at 14:00, and your training set has the frame from 14:00 while your validation set has the frame from 14:05, spatial-temporal data leakage occurs. The model memorizes the exact GPS coordinate of that mountain rather than learning the physical signature of combustion.

Split by Time / Year:

- **Train Set:** Summer fire seasons of 2020, 2021, and 2022.

- **Validation Set:** Summer fire season of 2023 (used to tune hyperparameters and early stopping).
- **Test Set:** Summer fire season of 2024 (kept locked away until the project is finished to test true real-world generalization).

2. Loss Function: Focal Tversky Loss

Do not use standard Binary Cross-Entropy (BCE). Use Focal Tversky Loss, which is specifically engineered for severe segmentation imbalance.

The Tversky index TI introduces constants α and β to control the trade-off between False Negatives (missed fires) and False Positives (false alarms):

$$TI = \frac{\sum p_i g_i}{\sum p_i g_i + \alpha \sum p_i(1 - g_i) + \beta \sum (1 - p_i) g_i}$$

Where p_i is predicted probability and g_i is ground truth (0 or 1).

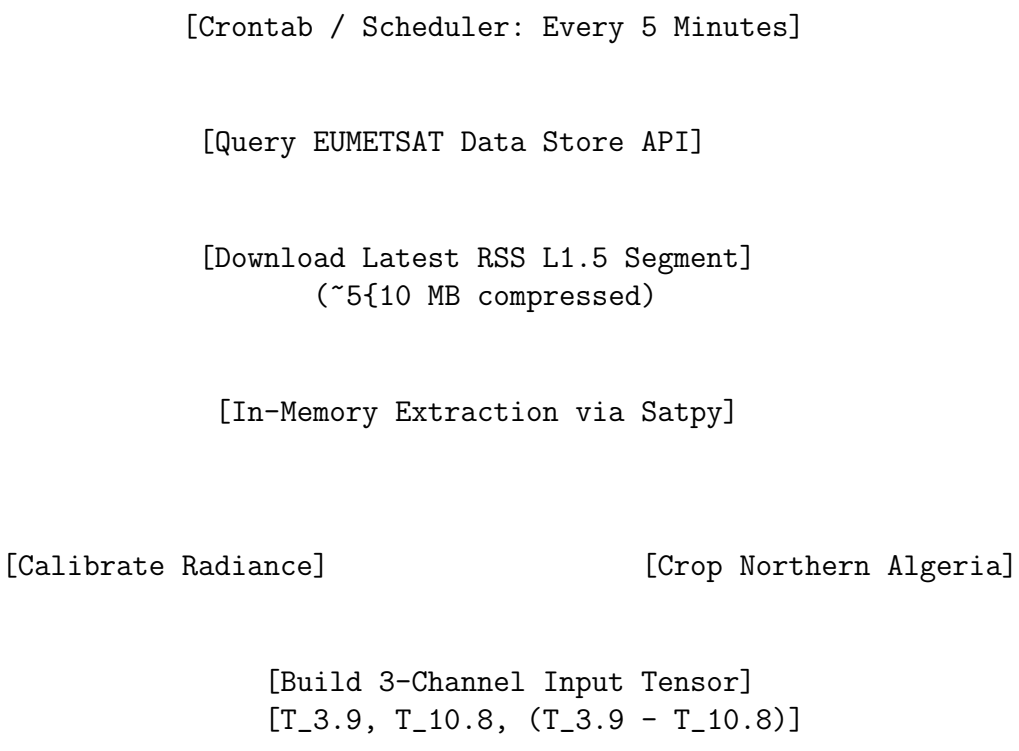
Setting $\beta = 0.7$ and $\alpha = 0.3$ penalizes False Negatives more heavily, forcing the network to prioritize catching fires even if faint.

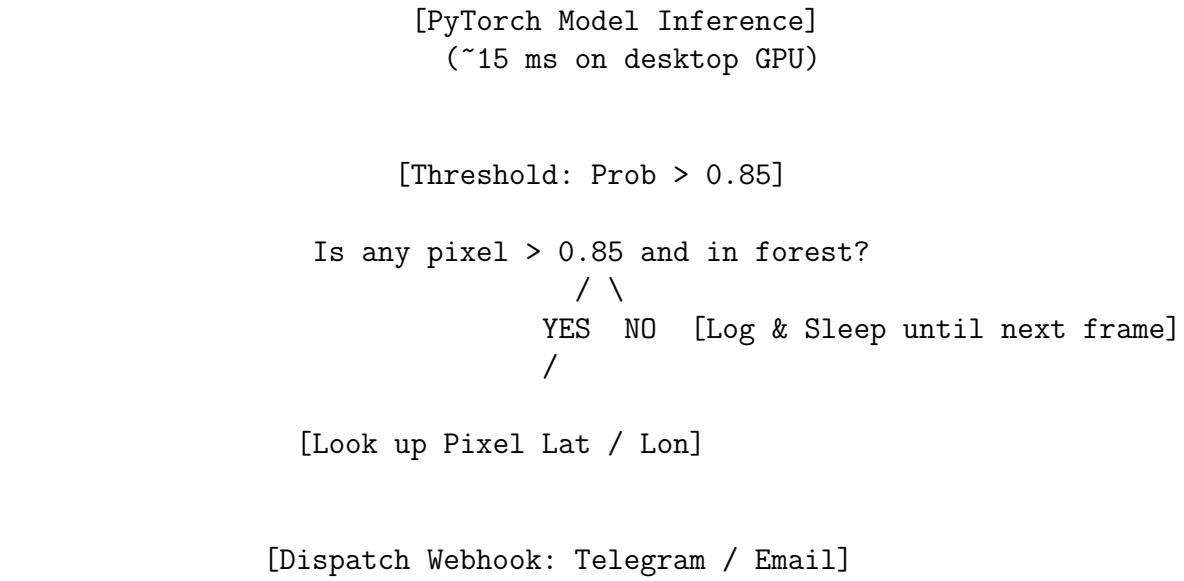
Adding a focal exponent $\gamma > 1$ focuses training attention on hard, ambiguous examples rather than easy background pixels:

$$\mathcal{L}_{\text{FocalTversky}} = (1 - TI)^{\frac{1}{\gamma}}$$

Phase 7: Real-Time Operational Pipeline

Once your model is trained and weights are frozen (`best_model.pth`), you deploy an autonomous runtime script.





The Live Execution Steps

1. **The Scheduler:** A lightweight background process (using Python `schedule` or a Linux `crontab`) triggers every 5 minutes, synchronized with EUMETSAT's publication clock.

2. **API Ingestion:** The script uses the official `eudataclient` or REST API to fetch only the newest Rapid Scan segment covering latitude 30°N −40°N.

3. **In-Memory Preprocessing:** Using `satpy`, the script reads the data directly from the binary stream without writing heavy intermediate files to disk:

   - Extracts 3.9 μm and 10.8 μm.
   - Crops to Northern Algeria bounding box (120×250).
   - Normalizes the bands using the exact mean and standard deviation calculated from your training set.

4. **Inference:** The tensor is pushed to your GPU:

```
with torch.no_grad():
    prediction = model(input_tensor) # Shape: [1, 1, 120,
       250]
    fire_mask = (prediction > 0.85).squeeze().cpu().numpy()
```

5. **False-Positive Masking:** Apply a static binary land mask: if a fire candidate falls in the open Mediterranean Sea or on bare Sahara sand dunes, discard it immediately.

6. **Alert Dispatch:** If valid fire pixels remain, convert their array indices (y, x) to real-world coordinates:

$$\text{Lat} = \text{Lat}_{\max} - y \cdot \Delta\text{Lat}, \quad \text{Lon} = \text{Lon}_{\min} + x \cdot \Delta\text{Lon}$$

   The script packages the coordinates into a message and sends it via a Telegram Bot API with an interactive map link.